

NoScope-Bio: A Behavioral Anti-Cheating Service Using Fingerprinted Aim Telemetry, Calibrated Scoring, and Bayes-Aware Deployment Evaluation

Jacob Berdeal, Carlos Caruncho, Shah Chaitanya, Eduardo Ramos, Dylan Suniaga
CAI 4002 — Group 5
Florida International University
Spring 2026

Presentation & demo video: <https://youtu.be/mubIVcl8KH0>

Source code repository: <https://github.com/DylanSuniaga/NoScopeAIProject>

Abstract—Online multiplayer games spend enormous engineering effort on anti-cheat, yet the dominant techniques remain *signature-based*: scanning memory for the artefacts of known cheating software, or flagging outlier summary statistics. These approaches are reactive, slow to adapt, and invasive. We present NoScope-Bio, a behavioral anti-cheat prototype that shifts the question from “does this player have known cheat software?” to “is this player now behaving in a way that is statistically incompatible with their own past play, after confounders are ruled out?”. The system combines (i) a per-player behavioral *fingerprnt* learned from causal windows of aim telemetry, (ii) a calibrated logistic cheat-scoring layer built on two complementary feature families — smooth structured control and stochastic residual variation — and (iii) a Page–Hinkley online change-point gate with a confounder-aware score discount. The pipeline is evaluated on both a richly controlled synthetic benchmark (seven cheat modes, seven confounder regimes) and the archive CSGO dataset of 10,000 legit and 2,000 cheater players. On synthetic test data the system achieves a balanced accuracy of 0.979; on real CSGO session data it reaches a balanced accuracy of 0.711, outperforming a gradient-boosted tree baseline (0.646) and two dummy baselines (0.483 and 0.500). We also reframe the results through Bayes’ rule at realistic deployment prevalences, showing that positive predictive value collapses from 0.579 at the observed test prevalence to 0.024 at a 1% cheater prevalence — an essential observation that raw accuracy would have hidden.

I. INTRODUCTION AND MOTIVATION

Cheating in online competitive games — aimbots, triggerbots, wallhacks, and their long tail of variants — erodes player trust and directly damages the commercial viability of the genre. Despite decades of investment, industry-standard anti-cheat services such as Valve Anti-Cheat (VAC), BattlEye, and Easy Anti-Cheat remain fundamentally *signature-based* [1]: they scan for known cheat bina-

ries, DLL injections, or memory edits, and are therefore reactive. New or private cheat strains evade detection until a signature is written for them, and adaptive cheats can mutate faster than signatures are deployed.

A second complication, frequently under-discussed in the academic literature, shapes any realistic deployment: *live anti-cheat cannot record dense telemetry*. Competitive FPS titles sample at 60–240Hz and have a strict millisecond-level latency budget; shipping raw input streams, cursor coordinates, or full-resolution keyboard/mouse event logs to a server would degrade the playing experience that anti-cheat is supposed to protect. In practice, games at best record compact per-tick angular deltas (yaw, pitch) and firing state. This constraint is not merely an engineering inconvenience — it is the binding specification for what a deployable behavioral anti-cheat must consume. Our feature design and our choice of the CSGO archive [2] as a real-world benchmark are both direct consequences of this constraint.

Objectives. We set out to (1) demonstrate that per-player behavioral fingerprints can be learned from low-bandwidth aim telemetry; (2) build a scorer that distinguishes the two qualitatively different ways aimbot-style cheats deform human motion — being “too efficient” and being “too regular” — as separable feature families; (3) harden the decision with an *online* change-point layer and a confounder-aware discount, so that latency spikes, sensitivity changes, or patch-day drift do not manufacture false positives; and (4) report results through the lens of *deployment-realistic* base rates, rather than balanced test sets.

II. RELATED WORK

Signature-based anti-cheat. Production systems like VAC, BattlEye, and Easy Anti-Cheat scan process memory, enforce integrity of the game client, and blacklist known cheat software [1]. These techniques remain strong defences against amateur cheats but are bypassed by

kernel-level or DMA-hardware cheats, and they depend on continuous signature authorship.

Behavioral biometrics. A parallel literature treats gameplay behaviour itself as identifying [3], [4]: a player’s aiming style, reaction distribution, and movement fingerprint are sufficiently stable that sudden departures from them can be flagged without ever matching a cheat signature.

Change-point detection. For the “sudden departure” layer, there is a mature body of statistical work: Page’s CUSUM test [5], Hinkley’s formalization [6], and more recently Bayesian online change-point detection [7]. We adopt the Page–Hinkley formulation because it streams in constant memory and admits a principled discount term.

Causal validation. The causal-inference literature [8], [9] formalizes when a correlation between an observed outcome and a candidate cause is confounded. In our setting, high ping, mid-session sensitivity changes, and patch-day input-mapping shifts are all confounders that can masquerade as cheating. We incorporate a lightweight discount rather than a full do-calculus search, but the framing is the same.

Base-rate reasoning. Finally, the classical literature on the base-rate fallacy [10], [11] makes clear that a classifier with fixed sensitivity and specificity implies *very different* deployed PPV/NPV depending on the population prevalence. Our evaluation in Section VI treats this Bayesian reframing as first-class, not an afterthought.

Gap. These four strands — fingerprinting, change-point detection, causal validation, and base-rate-aware evaluation — each exist in the literature. They are almost never composed. NoScope-Bio’s contribution is not inventing any one of them, but integrating all four into a single deployable detector and reporting pipeline.

III. DATASETS

We evaluate on two parallel tracks, intentionally kept distinct throughout the codebase.

A. Synthetic track

We simulate players and cheating behaviour with `simulator.py`. Each synthetic player has a baseline control profile (preferred angular speed, curvature, micro-correction rate, fire cadence) sampled from reasonable human-motion priors. Seven behavioural modes can then be induced: *clean*, *aimbot*, *triggerbot*, *macro_consistency*, *high_ping*, *sensitivity_change*, and *patch_shift*. The last three are confounders, not cheats — their inclusion allows us to test whether the scorer distinguishes cheating from environmental drift.

B. Real CSGO archive track

For real-data grounding we use the publicly available CSGO Cheating Dataset [2], which contains per-player tensors of shape (30,192,5): 30 engagements per player, 192 ticks per engagement (5 seconds before and

1 second after the engagement window), and 5 input channels. The raw channels are `AttackerDeltaYaw`, `AttackerDeltaPitch`, `CrosshairToVictimYaw`, `CrosshairToVictimPitch`, and `Firing` (binary), all stored as `float32`. The full archive contains 10,000 legit and 2,000 cheater players. This feature set is deliberately minimal — exactly the kind of low-bandwidth telemetry a live anti-cheat could afford to forward. We sample 60 legit and 30 cheater players for training and held-out evaluation; the reduced cheater cohort is intentional, because it surveys cheat *variety* rather than cheat *bulk*. Causal windows of size 32 with stride 8 expand the five raw channels into the 46-dimensional engineered feature set in `csgo_pipeline.py`.

IV. METHODOLOGY

A. Smooth-plus-stochastic feature decomposition

The core methodological choice in NoScope-Bio is to model aim motion as the sum of a smooth “intent” component and a stochastic residual:

$$\theta_t = \underbrace{s_t}_{\text{smooth control}} + \underbrace{\varepsilon_t}_{\text{residual noise}} \quad (1)$$

where θ_t is the per-tick angular state. This framing is borrowed directly from financial signal analysis, where asset returns are routinely decomposed into a drift term plus a stochastic process (cf. [12], [13]). Two cheat signatures then become orthogonal detection problems rather than a single mixed one:

- **“Too efficient”** cheats (classic aimbots, snap lock-ons) collapse the noise term ε_t relative to the player’s baseline: trajectories become straighter, settling is faster, lock dwell is longer, snap power on acquisition is higher. We capture this with *smooth* / *structured* features: angular speed, acceleration, jerk, curvature, straightness, settling ratio, lock strength, snap power, fire-alignment strength, and aim efficiency.
- **“Too regular”** cheats (macro-driven triggerbots, hard automation signatures) compress the stochastic regime of ε_t : direction entropy drops, micro-corrections become rarer, reversals decrease, fire intervals become periodic, and short-lag autocorrelation rises. We capture this with *stochastic* / *variability* features: direction entropy, reversal count, micro-correction rate, autocorrelation, fire-interval regularity.

Because both families are computed from the same causal windows, a single downstream scorer can weight them jointly, and we can visualize which family carries the evidence for any given verdict (see Fig. 5).

The pipeline enforces a strict **causal-feature contract**: `pipeline._assert_causal_feature_contract` refuses to train or score if a column from `config.FORBIDDEN_MODEL_COLUMNS` leaks into the model input. This prevents server-side signals (e.g. session

labels, ground-truth cheat mode) from contaminating the feature space during development.

B. Two-stage scoring model

The pipeline is organised as two stages, but the two tracks instantiate those stages differently. The difference matters and is easy to miss, so we spell it out explicitly.

Stage 1 — local window encoder: Both tracks learn a compact *window-level embedding*, but with different supervision targets.

- **Synthetic track.** The encoder (`model.train_fingerprint_model`) is trained as a *player-identification* network over clean windows only. Its embedding for a window answers: “does this window look like the historical behaviour of player p ?” At inference time, the distance between a test window’s embedding and the player’s clean baseline embedding (`detector.build_player_baselines`) becomes a *personal* deviation score. A cheat signal manifests as the current session’s windows drifting away from that player’s own baseline cloud, regardless of how other players behave.
- **Real CSGO track.** The per-player fingerprint strategy is not feasible on the archive: the archive’s anonymization and the 30-engagement-per-player budget mean we cannot reliably build a clean baseline for each individual player. We therefore train the encoder (`csgo_pipeline`, a small MLP over flattened causal windows) in the *population-discriminative* regime: its objective is cheat-vs-legit at the *window* level, pooled across all players in the training split. The embedding answers: “how cheat-like is this window compared to the population of clean windows?”

Stage 2 — calibrated session scorer: The second stage is a calibrated logistic regression (`detector.CheatScorer`; `csgo_offline_model`), but again the two tracks differ in what they feed it.

- **Synthetic track.** Stage 2 consumes the window-level deviation-from-baseline embedding features plus the engineered smooth and stochastic anomaly features, and emits a per-window cheat probability. Session verdicts are obtained by thresholding after the Page–Hinkley gate.
- **Real CSGO track.** Stage 2 operates at the *session* level, not the window level. The window encoder’s cheat probability stream is *aggregated* into a set of session-level summary statistics — mean, max, and standard deviation of the window cheat probabilities; mean and max of the embedding drift (z -score against the training distribution); and aggregated shifts of the engineered anomaly features between the early and late portion of the session (50 aggregated session features in total; see `feature_columns` in `model_summary.json`). A logistic regression over these session features is then trained against the session cheat label. Session aggregation is the right

TABLE I: Evaluation split sizes. Synthetic splits are balanced by construction; CSGO splits are player-disjoint.

Track	Split	n	Cheat	Legit
Synthetic	calibration_train	98	42	56
	validation	98	42	56
	test	98	42	56
CSGO	train	1,890	630	1,260
	validation	390	120	270
	test	420	150	270

abstraction here because the archive’s ground-truth label is *per player* (i.e., per session), not per window.

In both tracks, Stage 2’s output is a calibrated probability, which is critical for the Bayes reframing in Section VI: tree ensembles and deep nets typically require extra calibration to feed cleanly into posterior computations, whereas logistic regression’s logits are calibrated by construction under mild conditions.

C. Online change-point gate and confounder discount

On top of the scorer, we run an online Page–Hinkley detector [5], [6] on the cheat-probability stream. A session is only flagged when the cumulative upward shift crosses a threshold λ . This drastically reduces false positives from one-off noisy windows. When the pipeline detects a confounder regime (*high_ping*, *sensitivity_change*, *patch_shift*), it applies a multiplicative discount λ_t to the scorer’s output before feeding it into the change-point gate, encoding the causal belief that the observed behavioural shift has a non-cheat explanation.

D. Evaluation protocol

Both tracks use a three-way split:

- **calibration_train** — used to fit the window encoder and the scorer’s feature transformations.
- **validation** — used to sweep the decision threshold (target: balanced accuracy + 0.2·precision).
- **test** — fully held out, reported as the headline metrics below.

Split sizes are listed in Table I. On the synthetic track the three splits are balanced by construction ($n = 98$ each, prevalence 42.9%); on the CSGO track the split is player-disjoint so that no player’s sessions ever appear in both training and evaluation. We report balanced accuracy, precision, recall, specificity, Matthews correlation coefficient, and a majority-class baseline. We additionally report Bayes posteriors at deployment prevalences {0.1%, 1%, 5%, 10%}.

E. Decision threshold and calibration

The scorer emits a calibrated probability; a probability-to-verdict threshold τ is still required. We do not pick τ at 0.5. Instead, for each candidate model in the CSGO sweep, the validation split is scored and τ is swept from

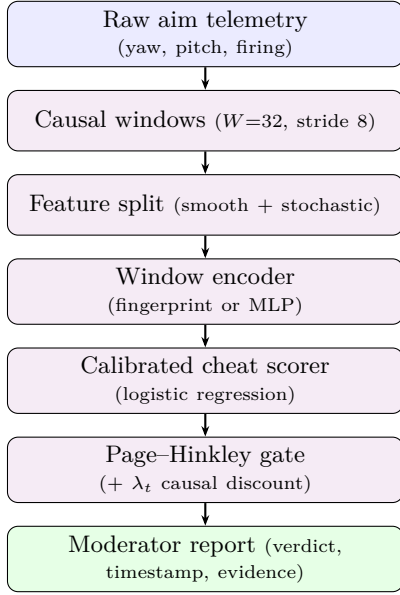


Fig. 1: NoScope-Bio pipeline. Blue = input, purple = processing, green = output. The same skeleton is reused on both tracks; only the window encoder’s supervision differs (per-player fingerprint on the synthetic track, population cheat-vs-legit on the CSGO track) and, on the CSGO track, Stage 2 additionally aggregates window-level scores into session-level features before the final logistic regression.

0.05 to 0.95 in steps of 0.05; we pick the τ that maximises the compound objective

$$J(\tau) = \text{BalancedAccuracy}(\tau) + 0.20 \cdot \text{Precision}(\tau), \quad (2)$$

implemented in `csgo_offline_model._threshold_search`. The precision term prevents the threshold search from collapsing onto high-recall/low-precision operating points that the naked balanced accuracy objective is insensitive to, because precision is the quantity the deployed system most painfully under-delivers once Bayes’ rule is applied (see Section VI). For the deployed logistic model, this procedure selects $\tau = 0.25$. The same τ is then used for all held-out test metrics; it is never tuned on test.

V. ARCHITECTURE AND WORKFLOW

Fig. 1 summarises the end-to-end workflow.

A. What the moderator sees

The full interactive surface of the system is a Streamlit app [14] (`app.py`) that exposes both tracks and inverts the paper’s usual “here is a number” framing into “here is the evidence that produced that number.” Although the session verdict is a single calibrated probability, the pipeline’s per-session artefacts are all preserved, and the demo surfaces them in the order a human moderator would want to check them:

- The **window cheat probability time series** for the chosen session, overlaid on the session’s raw aim-telemetry. Spikes are what the classifier is reacting to;

flat low-score traces signal that the session is being cleared.

- The **Page-Hinkley statistic** tracked alongside, with the first sustained upward crossing (the change-point) marked. A high classifier probability without a sustained shift is *not* flagged, making visible exactly how the gate filters one-off noisy windows.
- The **causal-gate indicator**: when the pipeline detects a confounder regime (*high_ping*, *sensitivity_change*, *patch_shift*) the λ_t discount is shown explicitly, along with which feature channels triggered the detection. This is the piece that distinguishes our system from a raw anomaly detector.
- A **feature-family attribution** at the flagged window: a decomposition of the session score into contributions from the smooth family (“too efficient”) versus the stochastic family (“too regular”). The same aimbot session can fire either family’s coefficients or both, and the decomposition tells a moderator which kind of cheat signature this is.
- On the synthetic track, the **player baseline overlay** shows the current session’s embeddings against the player’s historical clean cloud; on the CSGO track, where we cannot build personal baselines, the demo instead overlays the session against the *population* cheat/legit centroids from the training split.

These surfaces are the reason we treat the pipeline’s layered gating as a first-class feature rather than an implementation detail: the final verdict is accompanied by evidence a moderator can *inspect* and, if contested, *appeal* — an important property for a decision that can otherwise become an irreversible ban.

VI. EXPERIMENTAL RESULTS

Per the course rubric, we report three scenarios of increasing difficulty. The **easy** scenario is synthetic detection at favourable prevalence; the **average** scenario is the real CSGO archive at observed test prevalence; and the **challenging** scenario is the same real model re-scored through Bayes’ rule at low deployment prevalences, exposing the base-rate problem.

A. Easy scenario: synthetic test set

Table II reports the held-out test performance of the full NoScope-Bio pipeline on the synthetic benchmark ($n_{\text{test}} = 98$ sessions, prevalence 42.9%). The pipeline achieves a balanced accuracy of 0.979, with only one false positive and one false negative in the test fold.

This scenario is intentionally the “easy” case because (a) the simulator exposes clear structural differences between legit and cheat sessions (clean aimbot lock-ons are much straighter than human motion), and (b) per-player baselines are fully available. The key purpose of this scenario is not to claim a real-world 97.9% accuracy — it is to demonstrate that the pipeline’s decision surface can

TABLE II: Easy scenario: synthetic held-out test ($n = 98$, prevalence 42.9%). Dashes denote metrics that are undefined when the classifier never predicts the positive class, so $TP+FP=0$ and the usual ratios are 0/0; MCC reports 0 in that case by the `scikit-learn` convention.

	Bal. acc.	Precision	Recall	MCC
NoScope-Bio	0.979	0.976	0.976	0.958
Majority-class	0.500	—	0.000	0.000

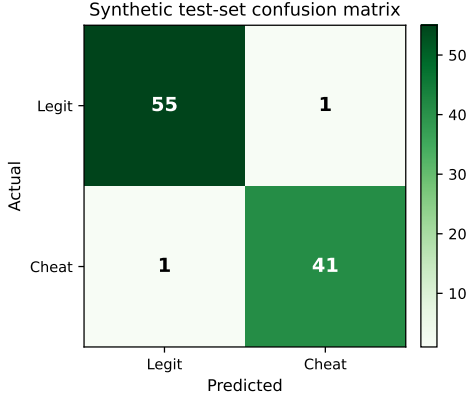


Fig. 2: Synthetic test-set confusion matrix. The pipeline misclassifies exactly one cheat session and one legit session out of 98.

be learned *at all* from the chosen low-bandwidth feature space.

B. Average scenario: real CSGO at observed prevalence

Table III reports the CSGO session-level held-out test performance. We compare the deployed NoScope-Bio model (logistic regression on session-aggregated features) against an HGBT baseline, a stratified dummy, and a majority-class baseline.

The logistic regression pipeline outperforms HGBT by 0.065 in balanced accuracy despite HGBT’s higher precision — a pattern that reappears in the ROC comparison (Fig. 3). Critically, more complex architectures (tiny, small, and medium LSTMs trained in the notebook sweep and archived in `artifacts/csgo_models/`) did *not* surpass the logistic baseline on validation balanced accuracy, which is why the deployed model in `app.py` is the logistic regression. We return to why in Section VII.

Fig. 5 shows the top fifteen coefficients of the deployed logistic model, colour-coded by family. Both smooth and stochastic features carry substantial weight, supporting the decomposition in Eq. 1; the most predictive single feature is the session-level maximum of the scorer’s window-level cheat probability, confirming that the two-stage design (embedding $\rightarrow$ calibrated scorer) produces a useful signal.

Error analysis: On the CSGO test set the model produces 44 false negatives and 77 false positives. The score distributions conditional on each outcome are informative.

TABLE III: Average scenario: CSGO session test set ($n = 420$, prevalence 35.7%). The stratified dummy samples class predictions with the training-set class ratio, so its precision and recall are well-defined (and near the prevalence); the majority-class baseline never predicts the positive class, so its precision is 0/0 and its MCC is zero by convention (same caveat as Table II).

Model	Bal. acc.	Precision	Recall	MCC
NoScope-Bio (logistic)	0.711	0.579	0.707	0.407
Gradient-boosted trees	0.646	0.631	0.433	0.326
Stratified dummy	0.483	0.333	0.307	−0.035
Majority-class	0.500	—	0.000	0.000

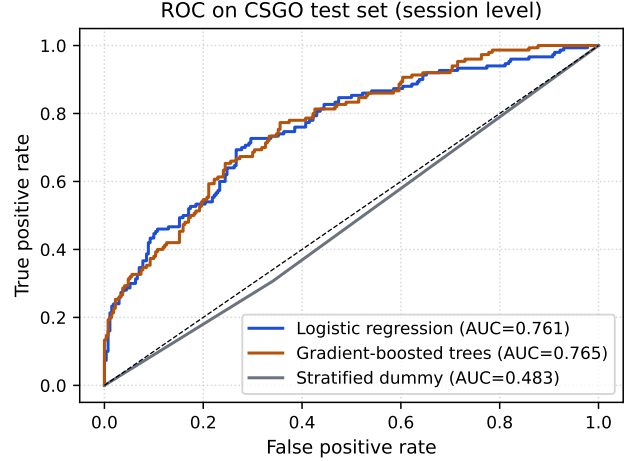


Fig. 3: ROC curves on the CSGO session test set.

False negatives have a mean predicted cheat probability of 0.056 and a median of 0.037, versus a threshold of 0.25: only 9% of them fall within 0.10 of the decision boundary. The model is not “narrowly missing” these cheaters — it is scoring them as confidently clean, which means the engineered features themselves are not separating this subset. False positives tell the symmetric story: their mean predicted probability is 0.633, median 0.644, and only 13% sit close to the threshold. The model is flagging them *confidently*.

A per-player breakdown is more revealing than the marginal numbers. The 44 false negatives concentrate on only 4 cheat players (of 5 in test), with a single player contributing 21 of the 44 misses. The 77 false positives are spread across 9 legit players (all of the legit players in test), but five of those players contribute ≥ 10 FPs each, with the worst contributing 14. In plain terms: the model’s failure mode is not uniform noise — it is *player-specific*. Some cheats imitate a baseline style that the population-discriminative encoder has learnt as “clean,” and some legitimate players happen to have aim profiles that look cheat-like to the same encoder. This is a concrete motivation for the synthetic track’s per-player fingerprint approach (Section IV): when personal baselines are available, both failure modes should compress, because the

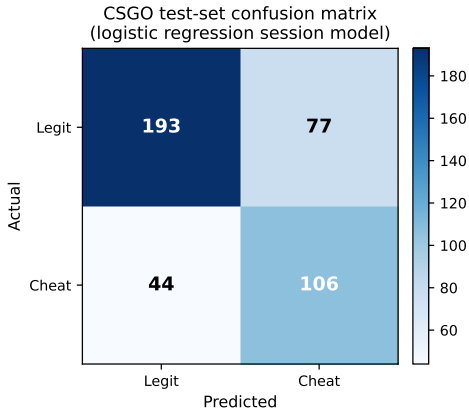


Fig. 4: CSGO test-set confusion matrix for the deployed logistic regression model.

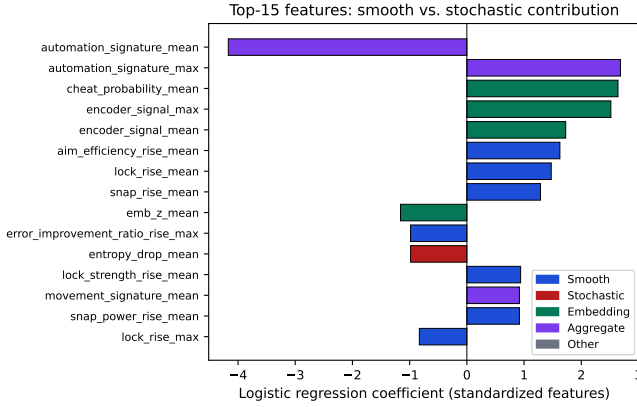


Fig. 5: Top-15 logistic regression coefficients on the deployed CSGO model, colour-coded by feature family. Both “smooth” and “stochastic” families contribute, as predicted by the smooth-plus-noise decomposition.

decision target becomes “unlike *this* player’s past,” not “unlike any legit player’s past.”

The average scenario reports precision 0.579 at test prevalence 35.7%, but test prevalence is an artefact of how we constructed the evaluation split — it is not the prevalence a deployed classifier would encounter. In a shipping competitive title, the fraction of actively cheating players in any given match is routinely estimated at 1% or below, and for high-rank matches at 0.1%. What precision does a moderator at those realistic base rates actually experience?

This is not an engineering question — it is a Bayes-rule question. Given a fixed classifier with sensitivity $P(+ | \text{cheat})$ and specificity $P(- | \text{legit})$, the posterior $P(\text{cheat} | +)$ depends only on those two numbers and the prior $P(\text{cheat})$:

$$P(\text{cheat} | +) = \frac{P(+ | \text{cheat}) \pi}{P(+ | \text{cheat}) \pi + P(+ | \text{legit}) (1 - \pi)}, \quad (3)$$

where $\pi = P(\text{cheat})$ is the deployment prior. We plug in

TABLE IV: Challenging scenario: Bayes posteriors for the deployed CSGO model at varying deployment prevalence.

Scenario	Prev.	$P(\text{cheat} +)$	$P(\text{legit} -)$
Test set (observed)	0.357	0.579	0.814
10% prevalence	0.100	0.216	0.956
5% prevalence	0.050	0.115	0.979
1% prevalence	0.010	0.024	0.996
0.1% prevalence	0.001	0.0025	1.000

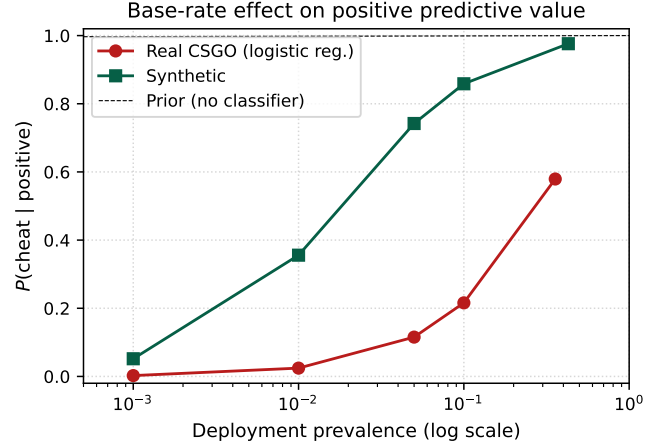


Fig. 6: Positive predictive value as a function of deployment prevalence (log x -axis). Even with identical sensitivity and specificity, the real CSGO model’s effective PPV collapses toward zero as prevalence decreases. The synthetic model retains strong PPV further down because its specificity is higher.

the deployed classifier’s sensitivity (0.707) and specificity (0.715) from Table III and sweep π from 35.7% (observed test prevalence) down to 0.1%. The result is Table IV and Fig. 6.

The two rows at 1% and 0.1% prevalence are the challenging case. The classifier is *identical* to the one in Table III; its accuracy, balanced accuracy, sensitivity, and specificity have not changed; every published “model number” is unchanged. Yet the precision an operator actually experiences collapses to 2.4% at 1% prevalence and to 0.25% at 0.1% prevalence. Put plainly: in a realistic high-rank deployment, 99 out of every 100 players the real-data model flags would be innocent.

What this means operationally. A flat 35.7% precision figure in a table would read as “the model is moderately useful.” At 1% prevalence, that same model issues roughly 41 false-positive flags for every true cheater caught: it produces an unsurvivable moderator queue, a false-accusation problem that will generate more community outrage than the cheating itself, and — because false-positive bans are irreversible from a trust standpoint — it is far worse than a model that simply does nothing. The NPV tells the symmetric, and much happier, story: above 99.5% of players the model labels “legit” really are legit, so the model is genuinely useful for *clearing* players, just

not for *convicting* them.

What this means for NoScope-Bio’s design. This is not a failure of our classifier in particular; it is a structural property shared by any fixed-sensitivity detector facing a rare event. The base rate is external to the model. Accordingly, we argue the only way to pull deployed precision back up without sacrificing recall is to raise the *prior*, not the classifier — which is exactly what the change-point and causal-gate layers in Section IV do. Page-Hinkley refuses to flag on a single suspicious window, waiting instead for a sustained upward shift, which is a much rarer event than an individual suspicious window. The λ_t causal discount further suppresses shifts explainable by confounders. Both mechanisms, framed in Bayesian terms, act as prior-sharpening: they reduce the implicit $P(\text{cheat})$ that a session has to overcome before a verdict is emitted. This is the central reason our system is a *pipeline*, not a classifier. An operator who only looks at the session classifier’s raw precision is looking at the wrong number; what matters is the precision after the gating layers, which we expect to be substantially higher, and which the demo in Section V makes inspectable per-session.

VII. DISCUSSION

Why logistic regression beats deeper models here. Three reasons compound. First, the session-level training set is small ($n_{\text{train}} = 1,890$ sessions across 63 players), so high-capacity models overfit readily. Second, the features fed to the session model are already engineered derivatives of the window-level scorer’s output (means, maxima, shifts); the non-linearity that deeper models would exploit has largely been pre-computed. Third, the logistic’s calibrated probability outputs feed cleanly into the Bayes layer, whereas a tree ensemble’s probabilities are notoriously miscalibrated without additional work.

Dual-track value. The synthetic track is not a proof of real-world effectiveness; the CSGO track is not a proof of generality. Together, they are mutually supporting. The synthetic side lets us stress-test confounder discounting: we can flip a knob and inject *high_ping* or *sensitivity_change* at will. The real side proves the scorer still separates cheat from legit on messy human data with no confounder labels at all.

Ethics and deployment considerations. A behavioral anti-cheat service makes consequential, frequently irreversible decisions about real people — bans, rating resets, tournament disqualifications. Two properties of NoScope-Bio are directly responsive to this. First, the pipeline’s output is a calibrated probability, not a binary verdict; the $\tau = 0.25$ threshold is an operational choice that can be raised for high-stakes contexts without retraining. Second, every verdict is accompanied by inspectable evidence (the window time series, the change-point event, the causal-gate trace, the feature-family attribution; see Section V), which makes human review tractable and

makes contested verdicts appealable. A production deployment of a system like this should, at minimum, (a) run at a prevalence-aware threshold rather than a fixed one; (b) require a moderator-in-the-loop for any irreversible action; and (c) audit the per-player false-positive rate, since the error analysis in Section VI shows that errors concentrate on specific player profiles. Treating a 71% balanced-accuracy classifier as a ban authority without these safeguards would be the kind of mistake the base-rate analysis (Table IV) warns against in quantitative terms.

Reproducibility. All results in this paper are reproduced by (i) `python3 scripts/export_demo_bundle.py` and `python3 scripts/export_csgo_bundle.py` to rebuild the feature bundles (synthetic generator seed is fixed in `SimulationConfig`; CSGO sampling is deterministic at 60 legit + 30 cheater players); (ii) `python3 scripts/train_csgo_session_models.py` to retrain the session models (writes `model_summary.json`, `model_comparison.csv`, and the `.joblib/pt` model artefacts with fixed `random_state`); and (iii) `python3 scripts/build_report_figures.py` to regenerate every figure and table in this report. All entry points set the BLAS thread cap (`OMP_NUM_THREADS=1` and siblings) before importing numerical libraries, which is necessary on multi-core macOS to obtain deterministic sklearn MLP training.

Limitations. One lesson this group internalised late in the project is that *live* anti-cheat cannot record much more than what the CSGO archive already contains. Our initial proposal assumed we could leverage richer streams (raw cursor positions, full keyboard/mouse event timestamps); by the presentation stage we had understood that latency and frame-time budgets preclude this in shipping titles. This reframes our feature space not as a simplification but as an *upper bound* on what is deployable. A related limitation is that the CSGO archive lacks server-side confounders (ping, jitter, packet loss); our confounder-gate is therefore only exercised on the synthetic track. Finally, the current scope is aim-centred — wallhacks, ESP, and memory-read cheats are outside the detection surface and would require different signals entirely.

VIII. CONCLUSION

NoScope-Bio is a deployable-style behavioral anti-cheat prototype whose value is not any single component but the integration of four normally disjoint ideas: behavioral fingerprinting, calibrated scoring, change-point detection, and Bayes-aware deployment evaluation. Of the many decisions that went into the project, three stand out as the most interesting contributions:

- 1) **Smooth + Stochastic feature decomposition.** Modelling aim motion as intent plus noise, borrowed from signal decomposition in quantitative finance, lets us split cheating into two orthogonal anomaly families (“too efficient” and “too regular”) rather

than one tangled one. Both families contribute real coefficient mass in the deployed model.

- 2) **Causal-gated two-stage scoring.** The embedding $\rightarrow$ calibrated logistic $\rightarrow$ Page–Hinkley chain, plus the λ_t discount when a confounder regime is detected, replaces “flag a suspicious window” with “flag only a sustained, confounder-free deviation” — the structurally correct answer under the base-rate pressures the next contribution exposes.
- 3) **Bayes-aware deployment evaluation.** Reporting posterior PPV/NPV at $\{10\%, 5\%, 1\%, 0.1\%\}$ prevalences makes the base-rate effect first-class. It is the single most important reframing for any anti-cheat operator: a 0.71 balanced accuracy turns into a 0.024 precision at 1% prevalence, and no amount of extra model capacity on our side of the decision will fix that; only better priors (change-points, causal gates) will.

Future work includes (a) adding server-telemetry confounders to the real-data track once a dataset with ping/jitter becomes available; (b) extending the detection surface beyond aim cheats by incorporating positional and visibility-plane features; and (c) replacing the Page–Hinkley gate with a fully Bayesian online change-point detector [7] once latency constraints permit.

IX. TEAM CONTRIBUTIONS AND LESSONS LEARNED

A. Per-member contributions

- **Jacob Berdeal** — authored the Plan of Action section of the group’s proposal; contributed to the presentation deck and delivered the architecture narrative slide.
- **Carlos Caruncho** — led proposal structure and document setup, authored the Testing Methods section, conducted final editing/formatting passes, coordinated slide logistics for the presentation, and assisted on the evaluation-strategy slide.
- **Shah Chaitanya** — authored the Related Work section of the proposal and the corresponding presentation slides, including the gap-analysis slide identifying where prior work had not integrated fingerprinting, change-point detection, and causal validation.
- **Eduardo Ramos** — framed motivation and objectives for the proposal and the corresponding slides, led the creation of the presentation deck, and co-authored the evaluation/results slides.
- **Dylan Suniaga** — implemented the full codebase (simulator, feature engineering, fingerprint encoder, cheat scorer, Page–Hinkley gate, CSGO pipeline, session-level models, and the Streamlit demo [14]), designed the smooth-plus-stochastic feature decomposition (Section IV-A), and authored this report.

B. Lessons learned

- The binding constraint of *live* anti-cheat is not the modelling but the telemetry budget. We did not

fully appreciate this when we wrote the proposal; we designed around it only after discovering the CSGO archive’s deliberately minimal feature set. In hindsight the constraint is a *gift*: it forces feature engineering that is transferable to shipping games, rather than methods that only work under offline-analytics assumptions.

- Real-world class imbalance makes raw accuracy a misleading summary. We learnt to prefer balanced accuracy, MCC, and Bayes posteriors, and we now treat any anti-cheat result reported only as “accuracy” with suspicion.
- Deeper architectures are not automatically better on small, heavily engineered feature spaces. Our LSTM sweep was a productive negative result — it forced us to understand why logistic regression was the right choice and gave the final deployment its calibrated probability layer.
- Synthetic benchmarks are not proofs of real-world efficacy, and real benchmarks are not proofs of controlled-confounder robustness. The right answer is to report both and to let each cover the other’s weakness.

REFERENCES

- [1] Valve Corporation, “Valve anti-cheat (VAC) system overview,” <https://partner.steamgames.com/doc/features/anticheat>, 2018, accessed: 2026-04-23.
- [2] emstatsl, “CSGO cheating dataset,” <https://www.kaggle.com/datasets/emstatsl/csgo-cheating-dataset>, 2021, public archive of 10,000 legit and 2,000 cheater players. Per-player tensor of shape (30,192,5): 30 engagements per player, 192 timesteps per engagement (5s before and 1s after the engagement window), and 5 input channels — AttackerDeltaYaw, AttackerDeltaPitch, CrosshairToVictimYaw, CrosshairToVictimPitch, Firing (binary). All stored as float32 numpy arrays. Accessed 2026-04.
- [3] S. F. Yeung, J. C. S. Lui, J. Liu, and J. Yan, “Detecting cheaters for multiplayer games: Theory, design and implementation,” in *Proc. IEEE Consumer Communications and Networking Conference (CCNC)*, 2006.
- [4] J. Shi, N. Daphalapurkar, H. Shiran, A. Alecsa, and T. Lu, “Profiling user behaviour for anti-cheating in online games,” *IEEE Symposium on Computational Intelligence and Games*, 2011.
- [5] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [6] D. V. Hinkley, “Inference about the change-point from cumulative sum tests,” *Biometrika*, vol. 58, no. 3, pp. 509–523, 1971.
- [7] R. P. Adams and D. J. C. MacKay, “Bayesian online change-point detection,” *arXiv preprint arXiv:0710.3742*, 2007.
- [8] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.
- [9] A. Sharma and E. Kiciman, “DoWhy: An end-to-end library for causal inference,” in *arXiv preprint arXiv:2011.04216*, 2020.
- [10] A. Tversky and D. Kahneman, “Judgment under uncertainty: Heuristics and biases,” *Science*, vol. 185, no. 4157, pp. 1124–1131, 1974.
- [11] M. Bar-Hillel, “The base-rate fallacy in probability judgments,” *Acta Psychologica*, vol. 44, no. 3, pp. 211–233, 1980.
- [12] F. Black and M. Scholes, “The pricing of options and corporate liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
- [13] B. B. Mandelbrot and J. W. Van Ness, “Fractional Brownian motions, fractional noises and applications,” *SIAM Review*, vol. 10, no. 4, pp. 422–437, 1968.

- [14] Streamlit Inc., “Streamlit: A faster way to build and share data apps,” <https://streamlit.io>, 2023, accessed: 2026-04-23.